

16-bit ALU

DDCO Assignment 4

NAME: CHAUDHARI DIPAK RAJENDRA

SRN: PES1UG24CS804

SECTION: K

The ALU (Arithmetic and Logic Unit) is the heart of a CPU since it performs the arithmetic and logic operations which are the key computations.

Your task in this assignment is to design and simulate a 16-bit¹ ALU which performs the following core operations:

- Arithmetic Addition and subtraction (two's complement)
- Logic And and or operations

Each operation would receive as input two 16-bit operands and the output would be a 16-bit result (along with carry and overflow). Which of the four operations are to be performed would be indicated by a two-bit operation input. So, the ALU module inputs are:

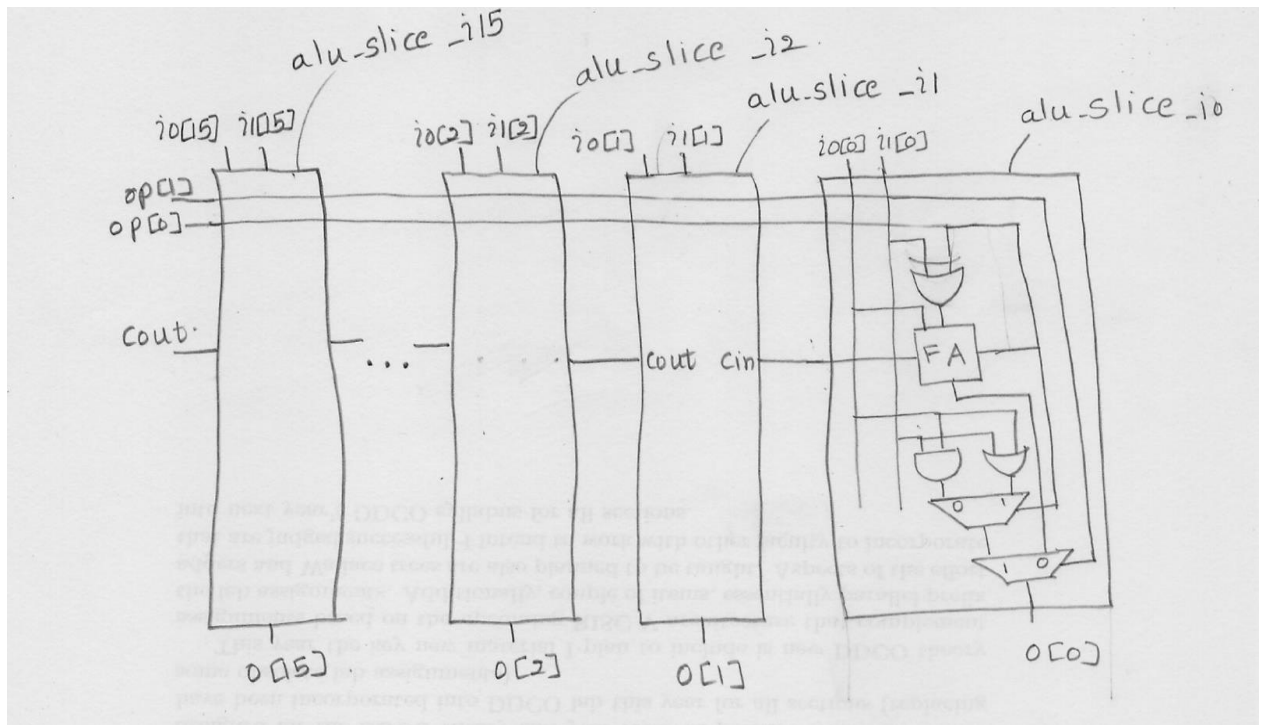
- i0[15:0] 16-bit operand 0
- i1[15:0] 16-bit operand 1
- op [1:0] 2-bit operation code

Operation code meaning:

Op [1:0]	operation
00	add
01	subtract
10	and
11	or

The ALU module outputs should be:

- **o [15:0]** 16-bit operation result
- Carry out of MSB
- Overflow Arithmetic overflow



CODES:

```

1  module alu_slice (
2      input wire i0, i1, Cin,
3      input wire [1:0] op,
4      output wire O, Cout
5  );
6      wire sum, and_out, or_out, xor_out, sub_i1, xor_i0_i1;
7      wire [3:0] mux_in;
8
9      and2 and_gate (i0, i1, and_out);
10     or2 or_gate (i0, i1, or_out);
11
12     xor2 xor_gate (i0, i1, xor_i0_i1);
13
14     // Full adder for ADD and SUB operations
15     wire sum_out, carry_out;
16     xor2 sum_xor (xor_i0_i1, Cin, sum_out); // Sum = i0 @ i1 @ Cin
17     and2 carry_and1 (i0, i1, carry_out);    // Carry-out logic for ADD
18     and2 carry_and2 (Cin, xor_i0_i1, Cout); // Carry-out logic continued
19
20     assign mux_in[0] = sum_out; // ADD result
21     assign mux_in[1] = sum_out; // SUB result (negated i1 is handled outside)
22     assign mux_in[2] = and_out;
23     assign mux_in[3] = or_out;
24
25     mux4 mux (mux_in, op[1], op[0], 0);
26
27 endmodule

```

```

1  module alu (
2      input wire [1:0] op,
3      input wire [15:0] i0, i1,
4      output wire [15:0] o,
5      output wire cout
6  );
7      wire [15:0] Cin, Cout;
8
9      // First slice (LSB), no carry-in
10     alu_slice slice_0 (
11         i0[0], .i1[0], 0, op, o[0], Cout[0]
12     );
13
14     genvar i;
15     generate
16         for (i = 1; i < 16; i = i + 1) begin : alu_slices
17             alu_slice slice (
18                 i0[i], .i1[i], Cout[i-1], op, o[i], Cout[i]
19             );
20         end
21     endgenerate
22
23     assign cout = Cout[15];
24 endmodule

```

GTKWAVE OUTPUT:

